

RESTful Web APIs & Architecture

REST principles, HTTP protocol mapping, REST vs. CRUD, status codes, and environment setup.

1. What is REST?

REST (Representational State Transfer) is an architectural style for designing networked applications. It relies on a stateless, client-server communications protocol—almost universally **HTTP**.

Statelessness

Every request from a client must contain all information necessary to understand and process it. No client state is stored on the server between calls.

Resource-Based

Resources (data entities like Users, Books, Orders) are identified using URIs and manipulated via standard HTTP methods.

Uniform Interface

Standardized representations (typically **JSON** or **XML**) decouple client implementations from server backend architectures.

2. Connecting REST Verbs to Database CRUD Operations

HTTP Method	CRUD Operation	Idempotent?	Primary Purpose & Behavior
GET	Read	Yes	Retrieves resource representation without modifying server state. Safe operation.
POST	Create	No	Creates a new resource on the server. Repeated calls create multiple entities.
PUT	Update / Replace	Yes	Replaces an existing resource entirely or creates it if it does not exist.
PATCH	Partial Update	No	Applies partial modifications to a resource (updates only supplied fields).
DELETE	Delete	Yes	Removes the specified resource entity from the server.

3. Key HTTP Status Codes Reference

2xx Success

- 200 OK** : Request succeeded.
- 201 Created** : Resource created successfully (via POST).
- 204 No Content** : Succeeded, returning no body (e.g., DELETE).

4xx Client Errors & 5xx Server Errors

- 400 Bad Request** : Malformed request payload.
- 401 Unauthorized** : Unauthenticated client.
- 403 Forbidden** : Authenticated but lacks permissions.
- 404 Not Found** : Target endpoint does not exist.
- 500 Internal Server Error** : Unhandled server exception.

4. Endpoint Naming Conventions: Good vs. Bad Routes

RESTful URIs should use **plural nouns** to represent resources. Actions should be dictated strictly by HTTP verbs, *never* by embedding verbs in URL paths.

Operation	✗ Poor / Non-RESTful Route	✓ Good RESTful Design
Get list of books	GET /api/getAllBooks	GET /api/books
Get single book detail	GET /api/getBookItem?id=5	GET /api/books/5
Create a new book	POST /api/createBook	POST /api/books
Update an existing book	POST /api/updateBook/5	PUT /api/books/5 or PATCH /api/books/5
Delete a book	GET /api/deleteBook/5	DELETE /api/books/5

5. Security: Authentication vs. Authorization

Security in RESTful services is split into two complementary phases:

Authentication ("Who are you?")

Verifies the identity of the user or client system making the request.

Common Approaches:

- HTTP Basic Authentication
- Token-Based Auth / Bearer Tokens
- JSON Web Tokens (JWT)
- OAuth 2.0 Third-Party Authentication

Authorization ("What can you do?")

Determines whether an authenticated entity has permission to perform a specific action on a resource.

Common Approaches:

- Role-Based Access Control (RBAC)
- Object-Level / Row-Level Permissions
- Scope / Token Permission Claims

6. RESTful Best Practices

- **Always Use JSON:** Use JSON as the standard data transport format for request bodies and response payloads. Set `Content-Type: application/json`.
- **Version Your APIs:** Include versioning directly in URIs (e.g., `/api/v1/books`) or via custom headers to allow breaking changes without disrupting existing clients.
- **Use Standardized Error Responses:** Return meaningful JSON error objects alongside correct HTTP error status codes.
- **Implement Filtering, Sorting & Pagination:** Do not return raw database tables; handle query parameters like `/api/books?page=2&limit=20&sort=-price`.

7. Planning & Organizing a Django API Project

A clean project layout separates API presentation logic, data serialization, and URL routing from standard server-rendered HTML views.

```
bookstore_project/ | ├── Pipfile / Pipfile.lock # Pipenv dependency management |  
manage.py | ├── bookstore/ # Global Settings | ├── settings.py # Installed apps (rest_framework),  
middleware | ├── urls.py # Root URL dispatcher | ├── books_api/ # Dedicated API App | ├── admin.py  
| ├── models.py # Book model schema | ├── serializers.py # Data conversion layer (Python <-> JSON)  
| ├── views.py # API view logic (handling GET, POST, etc.) | ├── urls.py # Endpoint paths  
(/api/v1/books)
```

8. First API View Example (Pure Django / JsonResponse)

```
from django.http import JsonResponse, HttpResponseNotAllowed from  
django.views.decorators.csrf import csrf_exempt from .models import Book import json @csrf_exempt  
def book_list_api(request): if request.method == 'GET': books = list(Book.objects.values('id',  
'title', 'author', 'price')) return JsonResponse({'books': books}, status=200) elif  
request.method == 'POST': data = json.loads(request.body) book = Book.objects.create(  
title=data['title'], author=data['author'], price=data['price'] ) return JsonResponse({'id':  
book.id, 'message': 'Book created'}, status=201) return HttpResponseNotAllowed(['GET', 'POST'])
```

9. Tools & Workflows for API Development

API Testing Tools

Postman / Insomnia / cURL: Craft HTTP requests, attach auth headers, inspect response JSON headers, and automate integration tests.

Browser Tools

DevTools (Network Tab): Monitor XHR/Fetch network calls, inspect request payloads, view response status codes, and analyze timing metrics.

Mock APIs

Use tools like JSON Server or Postman Mock Servers during frontend-backend parallel development before real endpoints are completed.